

Architecture Selection Justification

We chose Layered Architecture for the Smart Lab Equipment & Slot Reservation Portal System.

Architectural Choice

The system is organized into three layers: a Presentation Layer (User Interface Component), a Business Layer (Reservation Manager, Calibration Service, and Notification Service Components), and a Data Layer (Database Component). Each layer only communicates with the layer directly above or below it, keeping responsibilities clearly separated.

Two Reasons for This Choice

1. Predictable, moderate scale: The portal serves a single institution's lab equipment inventory rather than a high-traffic, multi-tenant workload. A Layered Architecture provides the structure needed without the operational overhead (service orchestration, distributed data consistency) that Microservices would introduce for a system of this size.
2. Clear separation for safety-critical logic: Calibration verification (FR-002) and slot locking (FR-001) are business rules that must be consistently enforced regardless of which interface (student portal or technician console) initiates the request. Isolating this logic in a dedicated Business Layer, rather than embedding it in the client as a simple Client-Server model might encourage, prevents rule duplication and inconsistent enforcement.

Security Advantage

Because the Data Layer is only accessible through the Business Layer, role-based access control (NFR-002) can be enforced centrally in the Reservation Manager and Calibration Service components. The User Interface Component never queries the database directly, which prevents a compromised or spoofed client from bypassing authorization checks to override reservation locks or falsify calibration records.

Performance Benefit

Concurrency control for slot reservations (NFR-001) is easier to guarantee when all booking requests pass through a single Reservation Manager component with direct, low-latency access to the Database Component. This avoids the network hops and eventual-consistency trade-offs that a Microservices approach would add, helping the system meet the 200ms concurrent-request target under peak load.